

James Stallkamp

Bram Lewis

CS 373

7/28/2024

Module 5 Write Up

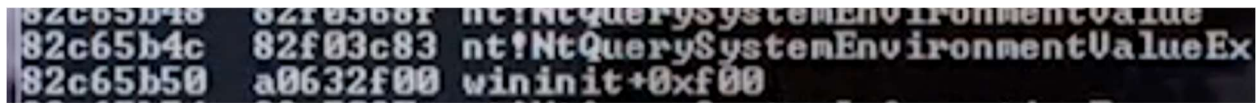
Introduction

Module 5 focused on exploring lower-level memory and components, specifically in the kernel space and how threads work with the CPU. A major form of malware that works on this level is called Rootkits, which I learned the actual definition is that a rootkit specifically tries to hide itself or files from the user in low level memory. Rootkits basically have a few major ways to infiltrate a system, bypassing driver sign checks, modifying windows boot path, kernel exploits, and stealing valid digi sigs. Basically, the idea is that the rootkit somehow places itself to intercept kernel memory which is makes sense because that is how it hides itself. A major concept that was introduced was the idea of kernel vs user mode memory, which can usually be identified as memory starting with 8 and 7 zeros, which might be important later.

During the week I looked at the Agony malware sample and used several tools to examine what the malware was doing. Using the analyzer, we found several interesting files that had changed their pointer location to try and hide a file that it had created. Using the KD tool we examined the actual memory and found the code that the rootkit had tried to replace. The top is the original and the bottom is the replaced code that will be launching the dangerous file.

```
82e4da0e 90          nop
kd> u 82e4da0f
nt!NtEnumerateValueKey:
82e4da0f 6a5c        push      5Ch
82e4da11 68c8cbc582  push      offset nt! ?? ::FNODOBFM::'string'+0x
cbc8>
82e4da16 e805f8e0ff  call     nt!_SEH_prolog4 (82c5d220)
82e4da1b 33db        xor       ebx,ebx
82e4da1d 895d94      mov      dword ptr [ebp-6Ch],ebx
82e4da20 6a08        push      8
82e4da22 59         pop       ecx
82e4da23 33c0        xor       eax,eax
kd> u a0633480
*** ERROR: Module load completed but symbols could not be loaded for w
wininit+0x1480:
a0633480 8bff        mov      edi,edi
a0633482 55         push     ebp
a0633483 8bec        mov      ebp,esp
a0633485 83ec24      sub      esp,24h
a0633488 56         push     esi
a0633489 57         push     edi
a063348a 0fb605004363a0 movzx    eax,byte ptr [wininit+0x2300 (a063430
a0633491 85c0        test     eax,eax
kd>
```

We also looked at the SSDT table which contains APIs that are available, one specific entry stood out because its memory address was not in the kernel space, which is a major red flag.



Lab 2

In lab 2 we looked at process memory using process hacker. Process hacker was able to show us all the memory that is allocated to the process including the “meat” of the process as well as everything that DLLs or other libraries have allocated. Windows implemented many restrictions on data in order to try and prevent the classic problem of separating data from instructions. This is interesting because as we say in previous slide fundamentally instructions are simply data itself, so this problem seems fundamentally not solvable. We saw a piece of malware called zBot that modified the process memory of the notepad application. It created many sections in memory with a name and had full access to do whatever it wants. These are obviously more red flags because all the other sections have meaningful names and tend to have restricted access based on their task.

Lab 3

For lab3 in the lectures we used winDBG on the nt!NTEnumerateValueKey and examined the actual instructions. Specifically, we looked for where a rootkit is intercepting instructions, looking at the call stack we found that “wininit” was added to the call stack.

After stepping through some of the labs we worked on patching damage from a rootkit. We learned that patching a rootkit is as simple as finding the modified pointers and patching them back to the original good pointer. To me rootkits have always seemed very dangerous, it is very interesting to me that the fix is as simple as restoring some pointers to reference the correct area in the kernel. Thinking through, it does make sense because the one thing that rootkits do which makes them so dangerous is modifying pointers to areas in kernel space. I do recognize that finding the pointers that were changed and restoring them to their original is not necessarily easy, but it does seem simple.

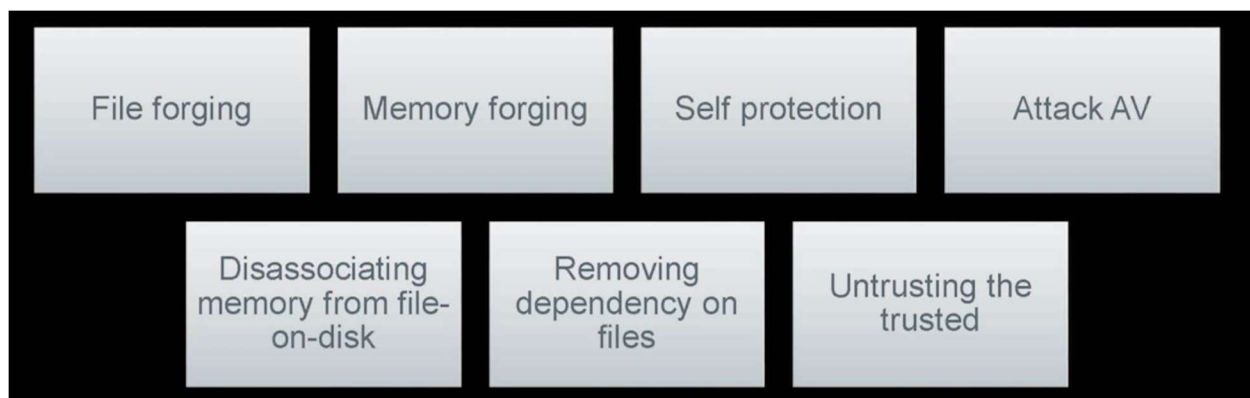
Modern RootKit techniques seemed interesting.

Inline hooks – HackerDefender – Zbot Import Table hooks – Adcliker-BA – Qoolaid	DKOM – TDSS Inline hook (Kernel) – Apropos IRP hook ▶ – TDSS, ZeroAccess SSDT hook – Ntrootkit-K	IDT hook – Apropos Sysenter hook – Spam-mailbot.c Filter driver – SearchNet – PigSearch MBR – Mebroot, TDSS
---	--	--

A major reoccurring avenue attack is to insert itself into the boot process and therefore bypass many security checks. This seems like a major advantage for stealth because the rootkit will be almost undetectable if it can hijack the boot process. To do this the malware injects itself into the MBR but maintains a good copy of the MBR so that it can still boot the system and make it appear healthy to the user. Microsoft has introduced a safe boot system where the digital sig is checked at every step of boot, again seems reactive. It was mentioned that booting off a second disk would essentially bypass the infected MBR and prevent the rootkit from hiding as well, this is a very useful tool for finding rootkits.

We then looked at a file called TDSS and watched it changed the values in the MBR. Before running the bootKit we did save a copy of the first section of the disk using “sectEdit”.

Modern trends for rootkits to hide themselves. The overwhelming theme of rootKits seems to be to hide itself, maybe more so than other types of malwares. It seems like rootkits could in theory operate in the background of your system and steal data forever if it is able to remain hidden, hence the importance of stealth.



Conclusion

During this week we focused on various areas of windows memory but specifically how attackers can access kernel memory. Attackers have many ways of inserting code that can hijack

pointers to kernel space allowing them to use low level system calls. RootKits seem very focused in hiding themselves from the user, this makes sense because in theory then the rootkit would stay on the system forever. A reoccurring method for rootKits was identifying red flags and simply watching for other suspicious activity. My immediate concern is that this seems highly reactive not proactive, IE how do we get ahead of attackers rather than simply responding to their creativity. The speaker did address this that there are heuristics to try and detect similar virus patterns but I have to admit I found the response lacking.

Appendix

This write-up follows the MLA standard for expository essays.